

XPBF: EXTENDED POSITION BASED FLUID

Start from Explicit Euler Integration

Position Based Dynamics (PBD), introduced by *Matthias Müller*, is a geometric formulation of constrained dynamics where unknown constraint forces are not solved explicitly, but instead replaced by direct position-level constraint enforcement. It is motivated by the problems of explicit Euler integration of Newton’s second law, which introduces numerical drift and requires explicit handling of constraint forces. Instead, the system evolves under external forces using standard integration, and constraint effects are implicitly handled as geometric projections onto constraint manifolds defined by holonomic equations.

Consider a pendulum modeled as a particle under gravity. In a force-based formulation, numerical integration leads to constraint drift and violation of the fixed-length condition, producing spring-like behavior. PBD instead enforces

Lagrange Multiplier Method

The Lagrange multiplier method is a technique for *finding stationary point of a function subject to constraints* by introducing an auxiliary variable λ to enforce the constraint. It was developed by *Joseph-Louis Lagrange* in the 18th century as part of his work on mechanics and calculus of variations. The method converts a constrained problem into solving $\nabla_x \mathcal{L} = \nabla f + \lambda \nabla g = 0$ together with $\nabla_\lambda \mathcal{L} = g(x) = 0$.

To update x' to satisfy the constraint,

we want to move x' with the minimal kinematic energy f :

$$f = \min_v \frac{1}{2} v^T M v, \text{ where } v = \begin{bmatrix} v_1 \\ v_2 \\ \dots \\ v_n \end{bmatrix} = \dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dots \\ \dot{x}_n \end{bmatrix}.$$

So we build Lagrangian function with constraint $g = C(x')$:

$$\mathcal{L} = f + \lambda g,$$

write it in discrete form:

$$x' \rightarrow x + \Delta x,$$

$$f(\Delta x) = \frac{1}{2} \sum_i \Delta x_i^T m_i \Delta x_i,$$

$$g(\Delta x) = \lambda C(x + \Delta x),$$

approximate C by first-order discrete expansion (Taylor):

$$C(x + \Delta x) \approx C(x) + \nabla C(x) \cdot \Delta x = C(x) + \sum_i \nabla_{x_i} C(x) \cdot \Delta x_i,$$

treat C as a constant because x is the current positions.

Gauss Seidel Method

PBD adopts a Gauss Seidel iteration, where constraints are processed sequentially and each correction is applied immediately to the system state. *Instead of solving constraint space globally, subsequent constraints operate on already updated positions*, allowing information to propagate through the system within a single iteration. This iterative projection process converges toward a configuration that satisfies all constraints, but *the convergence process itself introduces an implicit geometric behaviour similar to stiffness*. As the number of iterations increases, constraints appear increasingly rigid, since repeated projections progressively reduce constraint violations.

XPBD:Extended Position Based Dynamics

In PBD, constraint stiffness is not an explicit physical parameter but emerges from the numeric solution. In other words, PBD does not solve time-continuous dynamics but instead aiming at a constraint-satisfying configuration as the limit of discrete projection iterations, where convergence — rather than intermediate states — is the target, which is why the iteration is independent to time step. *Extended Position Based Dynamics (XPBD)*, proposed by *Miles Macklin*, resolves this limitation by introducing a *compliance term into length constraints* (the most used constraint) formulation and incorporating an *accumulated Lagrange multiplier*. This results in a modified constraint equation that decouples stiffness from the iterative solver, yielding physically interpretable constraint behavior.

Now we extend the previously introduced explicit Euler integration to constrained mechanical systems by incorporating the constraint as forces:

$$M\ddot{x} = f + J^T \lambda, \quad J = -\nabla_x C(x)$$

J : constraint Jacobian

λ : unknown constraint force magnitude

$\mathcal{C}(x) = \|x - x_0\|^2 - r^2 = 0$ by projecting the particle back onto the circular constraint after each update using gradient-based position corrections.

$$M\ddot{x} = f$$

$$v' = v + \Delta t M^{-1} f$$

$$x' = x + \Delta t v'$$

$$x' = x + \Delta t v + \Delta t^2 M^{-1} f$$

Then PBD adds:

$$\mathcal{C}(x') = 0$$

Finally we get the Lagrangian function:

$$\mathcal{L}(\Delta x, \lambda) = \lambda C + \sum_i (\frac{1}{2} \Delta x_i^T m_i \Delta x_i + \lambda \nabla_{x_i} C \cdot \Delta x_i),$$

where $\mathcal{L}$ w.r.t Δx is convex quadratic, and $\mathcal{L}$ w.r.t λ is linear.

So $\mathcal{L}$ has saddle-like shape with an only minimal value at the points where $\nabla \mathcal{L} = 0$.

Now we can safely solve:

$$\nabla_x \mathcal{L} = \nabla f + \lambda \nabla g = 0 \text{ and } \nabla_\lambda \mathcal{L} = g(x) = 0.$$

For each particle x_i ,

$$\nabla_{x_i} \mathcal{L} = \frac{\partial}{\partial x_i} (\frac{1}{2} \Delta x_i^T m_i \Delta x_i + \lambda \nabla_{x_i} C \cdot \Delta x_i) = m_i \Delta x_i + \lambda \nabla_{x_i} C,$$

$$\nabla_{x_i} \mathcal{L} = 0 \implies m_i \Delta x_i + \lambda \nabla_{x_i} C = 0 \implies \Delta x_i = -m_i^{-1} \nabla_{x_i} C \lambda.$$

Plug in $\nabla_\lambda \mathcal{L}$,

$$\nabla_\lambda \mathcal{L} = C + \sum_i \nabla_{x_i} C \cdot \Delta x_i = C - \sum_i \nabla_{x_i} C \cdot m_i^{-1} \nabla_{x_i} C \lambda,$$

$$\nabla_\lambda \mathcal{L} = 0 \implies C - \sum_i \nabla_{x_i} C \cdot (m_i^{-1} \lambda \nabla_{x_i} C) = 0 \implies \lambda = \frac{C}{\sum_i m_i^{-1} \|\nabla_{x_i} C\|^2}.$$

$$v' = v + \Delta t M^{-1} (f + J^T \lambda),$$

$$x' = x + \Delta t v' = (x + \Delta t v + \Delta t^2 M^{-1} f) + \Delta t^2 M^{-1} J^T \lambda = x'_f + \Delta t^2 M^{-1} J^T \lambda$$

In PBD, $x' = x + \Delta x$, $\Delta x = M^{-1} J^T \lambda$, where Δt^2 is ignored.

In XPBD, it adds Δt^2 back by redefining the Lagrangian function:

$$f(\Delta x) = \frac{1}{2} \sum_i (\frac{\Delta x_i}{\Delta t})^T m_i \frac{\Delta x_i}{\Delta t}.$$

In PBD, $g(\Delta x) = C(x + \Delta x)$, which means rigid geometric projection.

In XPBD, $\lambda = k C(x + \Delta x)$, which extends length constraints to Hooke’s law:

$$g(\Delta x) = C(x + \Delta x) - k^{-1} \lambda.$$

$$\mathcal{L}(\Delta x, \lambda) = \frac{1}{2} \sum_i (\frac{\Delta x_i}{\Delta t})^T m_i \frac{\Delta x_i}{\Delta t} + \lambda C + \lambda \nabla_{x_i} C \cdot \Delta x_i - k^{-1} \lambda^2, \quad \nabla \mathcal{L} = 0.$$

$$\text{Finally we get: } \Delta x_i = -\Delta t^2 m_i^{-1} \nabla_{x_i} C \lambda, \quad \lambda = \frac{C}{\Delta t^2 \sum_i m_i^{-1} \|\nabla_{x_i} C\|^2 + 2k^{-1}}.$$

Implicit Euler Integration

In implicit Euler method, future state is evaluated.
future force rather than current force is used to solve the system:

$$v' = v + \Delta t M^{-1} f(x')$$

$$x' = x + \Delta t v' = x + \Delta t v + \Delta t^2 M^{-1} f(x') = x_p + \Delta t^2 M^{-1} f(x')$$

In an XPBD loop, we expect x' to converge to the future state, by constraint:

$$C(x') - k^{-1} \lambda_n = 0 \implies C(x_n + \Delta x_n) - k^{-1} \lambda_n = 0.$$

This satisfies the future state locally by one step, pulling $x_n \rightarrow x'$.
But because of Gauss Seidel, other constraints may pull particles away to x_{n+1} .
At next step, we want λ to pull $x_{n+1} \rightarrow x'$ (i. e., $x_n + \Delta x_n$).

The new constraint becomes:

$$C' = (C(x_{n+1} + \Delta x_{n+1}) - C(x_n + \Delta x_n)) - k^{-1} \lambda_{n+1} = 0$$

$$\implies C(x_{n+1} + \Delta x_{n+1}) - k^{-1} (\lambda_n + \lambda_{n+1}) = 0.$$

The new correction only compensates:
the remaining deviation from the previously satisfied state.

Smoothed Particles Hydrodynamics

Smoothed Particle Hydrodynamics (SPH) is a mesh-free Lagrangian method for simulating fluids in which a continuous medium is represented by particles and field quantities are reconstructed using smoothing kernels. In SPH, any physical quantity is approximated as a weighted sum over neighboring particles: $A(x_i) = \sum_j m_j \frac{A_j}{\rho_j} W(x_i - x_j)$, where W is a compactly supported kernel function. A key quantity is density, computed as $\rho_i = \sum_j m_j W(x_i - x_j)$, which allows pressure to be defined via an equation of state $p_i = k(\rho_i - \rho_0)$. Fluid motion is then obtained by evaluating forces such as pressure forces $F_i^{pressure} = - \sum_j m_j \frac{p_i + p_j}{2\rho_j} \nabla W(x_i - x_j)$, leading to particle accelerations that approximate the Navier–Stokes equations in a fully particle-based form.

Extended Position Based Fluid Dynamics

Position-Based Fluids formulates incompressible fluid simulation as a constraint projection problem on particle positions, rather than solving pressure forces explicitly. The method is built upon Smoothed Particle Hydrodynamics (SPH) density estimation and enforces incompressibility through iterative position corrections derived from Lagrange multipliers. In Position Based Fluids, the authors observe:

1. Density Constraint via SPH

The local density constraint at particle i is computed using a smoothing kernel:

$$C_i = \frac{\rho_i}{\rho_0} - 1,$$

$$\rho_i = \sum_j m_j W_{ij},$$

$$W_{ij} = W_{poly6}(r_{ij}),$$

$$r_{ij} = x_i - x_j,$$

$$W_{poly6}(r, h) = \frac{315}{64\pi h^9} (h^2 - \|r\|^2)^3.$$

2. Lagrangian Multiplier via Gradient of Constraint

Based on XPBD formula: $\Delta \lambda = \frac{-C - \beta \alpha \lambda}{m_i^{-1} \sum_i \|\nabla_{x_i} C\|^2 + \alpha}$, β : reinforcement rate

The constraint gradient is required to determine the Lagrangian multiplier.

For each constraint C_i ,

$$\nabla C_i = \sum_k \nabla_{x_k} C_i = \nabla_{x_i} C_i + \sum_j \nabla_{x_j} C_i,$$

Apply the chain rule:

$$\nabla_{x_i} C_i = \frac{1}{\rho_0} \nabla_{x_i} \rho_i, \quad \nabla_{x_j} C_i = \frac{1}{\rho_0} \nabla_{x_j} \rho_i,$$

Compute derivative w.r.t particle i :

$$\nabla_{x_i} \rho_i = \sum_j m_j \nabla W(x_i - x_j) = \sum_j m_j \nabla W_{ij},$$

$$\nabla_{x_j} \rho_i = - m_j \nabla W(x_i - x_j) = - m_j \nabla W_{ij} = m_j \nabla W_{ji},$$

$$\nabla W = \nabla W_{spiky}(r, h) = - \frac{45}{\pi h^6} (h - r)^2 \frac{r}{\|r\|}.$$

It forces $C(x_{n+1} + \Delta x_{n+1})$ to match $C(x_n + \Delta x_n)$ with a smaller λ_{n+1} .

To simplify the expressions,

$$\text{let } \mu = -\Delta t^2 \lambda, \quad \alpha = \frac{2k^{-1}}{\Delta t^2}$$

$$\implies \nabla_{\mu} \mathcal{L} = C + \sum_i \nabla_{x_i} C \cdot \Delta x_i + \alpha \mu.$$

$$\implies \Delta x_i = m_i^{-1} \nabla_{x_i} C \mu, \quad \mu = - \frac{C}{\sum_i m_i^{-1} \|\nabla_{x_i} C\|^2 + \alpha},$$

To avoid over correction,

$$\mu \rightarrow \mu + \Delta \mu,$$

$$\nabla_{\Delta \mu} \mathcal{L} = C + \sum_i \nabla_{x_i} C \cdot \Delta x_i + \alpha (\mu + \Delta \mu) = 0,$$

$$\implies \Delta \mu = \frac{-C - \alpha \mu}{\sum_i m_i^{-1} \|\nabla_{x_i} C\|^2 + \alpha}, \quad \Delta x_i = m_i^{-1} \nabla_{x_i} C \Delta \mu$$

$\Delta \mu$: current force magnifier
 μ : previous force magnifier

This is the final XPBD formula presented in the paper.

3. Position Correction via Lagrangian Multiplier

Based on XPBD formula: $\nabla_{\lambda} \mathcal{L} = C + \sum_i \nabla_{x_i} C \cdot \Delta x_i + \alpha (\lambda + \Delta \lambda) = 0$

Each particle i is affected by itself C_i and its neighbors: $C_{j_1}, C_{j_2} \dots C_{j_n}$.

Here we apply Gauss Seidel method, solving C_i, C_j sequentially:

$$C_i + \sum_k \nabla_{x_k} C_i \Delta x_k^i + \alpha \lambda_i = 0,$$

↓

$$x_i \leftarrow x_i + \Delta x_i^i, \quad i \in k$$

↓

$$C_j + \sum_k \nabla_{x_k} C_i \Delta x_k^j + \alpha \lambda_j = 0,$$

↓

$$x_i \leftarrow x_i + \Delta x_i^j, \quad i \in k$$

..

So, we finally get:

$$\Delta \lambda_i = - \frac{\frac{1}{\rho_0} \sum_j m_j W_{ij} - 1 + \beta \alpha \lambda_i}{\frac{1}{\rho_0^2} \left(m_i^{-1} \left\| \sum_j m_j \nabla W_{ij} \right\|^2 + \sum_j m_j^{-1} \|m_j \nabla W_{ij}\|^2 \right) + \alpha}$$

$$\Delta \lambda_j = - \frac{\frac{1}{\rho_0} \sum_k m_k W_{jk} - 1 + \beta \alpha \lambda_j}{\frac{1}{\rho_0^2} \left(m_j^{-1} \left\| \sum_k m_k \nabla W_{jk} \right\|^2 + \sum_k m_k^{-1} \|m_k \nabla W_{jk}\|^2 \right) + \alpha}$$

$$\Delta x_i^i = m_i^{-1} \nabla_{x_i} C_i \lambda_i = m_i^{-1} \left(\frac{1}{\rho_0} \sum_j m_j \nabla W_{ij} \right) \Delta \lambda_i,$$

$$\Delta x_i^j = m_i^{-1} \nabla_{x_i} C_j \lambda_j = m_i^{-1} \left(- \frac{1}{\rho_0} m_i \nabla W_{ji} \right) \Delta \lambda_j = m_i^{-1} \left(\frac{1}{\rho_0} m_i \nabla W_{ij} \right) \Delta \lambda_j,$$

$$\Delta x_i = \Delta x_i^i + \sum_j \Delta x_i^j = m_i^{-1} \frac{1}{\rho_0} \sum_j \nabla W_{ij} (m_j \Delta \lambda_i + m_i \Delta \lambda_j).$$

Artificial Pressure for Surface Tension Correction

In Position Based Fluids, the authors observe: When particles get very close, the density constraint gradients become too small. The solver then fails to separate particles. This leads to pairing instability (clumping). So they add an artificial pressure term to prevent particle clustering.

They defined:

$$\Delta x_i = m_i^{-1} \frac{1}{\rho_0} \sum_j \nabla W_{ij} (m_j \Delta \lambda_i + m_i \Delta \lambda_j + s_{corr}),$$

$$s_{corr} = -k \left(\frac{W(r_{ij})}{W(\Delta q)} \right)^n.$$

k: small constant (e.g. 0.001)

n: typically 4~8

Δq: a small reference distance (e.g. 0.1 × smoothing radius)

When particles are close $\Rightarrow \frac{W(r_{ij})}{W(\Delta q)} \approx 1 \Rightarrow$ strong effect.

When particles are far apart $\Rightarrow \frac{W(r_{ij})}{W(\Delta q)} \rightarrow 0 \Rightarrow$ negligible effect.

Exponent (·)ⁿ increases non-linearity and makes effect sharply drop off.

Negative strength −k ensures repulsion and pushes particles apart.

Numerically, this creates a nonlinear response: very small separations lead to large corrections (preventing clustering), moderate distances yield negligible correction (normal fluid behavior), and beyond the kernel support radius the contribution is exactly zero, ensuring stability and locality of the simulation.